University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

Zuverlässigkeit verteilter Datenketten in der Fertigung

Fehlermodell, Messung und Architekturvergleich einer MQTT-basierten IIoT-Datenkette

Exposé und Antrag auf Durchführung einer Masterarbeit
in Kooperation mit der ProSeS GmbH
Master Studiengang Informatik

Autor: Erlind Sejdiu
Matrikelnummer: 106357

Betreuer Hochschule: [Name]
Betreuer Unternehmen: [Name, Abteilung]
Geplanter Zeitraum: [TT.MM.JJJJ] – [TT.MM.JJJJ] (6 Monate)

Stand: 2. November 2026

Inhaltsverzeichnis

1 Motivation **1**

2 Problemstellung **1**

2.1 Das System als verteilte Datenkette 1

2.2 Was MQTT garantiert und was nicht 1

2.3 Was die Verbraucher brauchen 2

2.4 Zuverlässigkeit als Bewertungsrahmen 2

2.5 Die Lücke 3

3 Forschungsfragen und Ziele **3**

4 Nutzen für die ProSeS GmbH **4**

5 Rahmenbedingungen **4**

6 Zeitplan **5**

Literaturverzeichnis **5**

1 Motivation

Die industrielle Prozessdatenerfassung über Message Queuing Telemetry Transport (MQTT) bildet das nachrichtentechnische Rückgrat moderner Architekturen für das Industrial Internet of Things (IIoT) [1]. Erzeuger veröffentlichen ihre Daten auf einem Broker, und Verbraucher abonnieren genau die Daten, die sie brauchen. Diese lose Kopplung ist der Grund für die Verbreitung von MQTT, denn Erzeuger und Verbraucher müssen sich nicht kennen [2].

In der Praxis wird MQTT selten auf der grünen Wiese eingeführt, sondern in eine bestehende Systemlandschaft eingebaut. Fertigungsdaten durchlaufen dann eine Kette von Komponenten: Sie werden an der Maschine ausgelesen, gespeichert, weitergeleitet und schließlich angezeigt oder für die Planung verwendet. Jede Komponente läuft als eigener Prozess, oft auf eigenen Rechnern, und kann unabhängig von den anderen ausfallen. Das Ganze ist damit ein verteiltes System [3]. Die Verbraucher am Ende der Kette, etwa eine Hallenanzeige oder eine Feinplanung [4], brauchen ein korrektes und aktuelles Bild der Fertigung. Geht eine Meldung unterwegs verloren, kommt sie doppelt oder zu spät an, zeigt die Halle einen falschen Maschinenstatus, oder die Planung rechnet mit falschen Stückzahlen.

MQTT sichert die Zustellung nur zwischen einem Client und dem Broker ab, nicht von Ende zu Ende [5]. In einer Kette aus mehreren Komponenten gilt das für jede Übergabe einzeln: Selbst wenn jede Übergabe für sich zuverlässig ist, folgt daraus keine Zuverlässigkeit von der Maschine bis zum Verbraucher [6]. Was nach dem Neustart einer Komponente, einem Netzwerkausfall oder einer nicht erreichbaren Datenbank mit den Daten passiert, muss die Architektur selbst regeln. Erweiterungen wie Sparkplug B [7] und Muster aus verteilten Systemen wie Event Sourcing [8] oder Log-basierte Plattformen [9] bieten dafür Lösungen, jeweils mit eigenen Annahmen und Kosten.

Bei der ProSeS GmbH wurde MQTT erst vor Kurzem eingeführt, und weitere Komponenten wie eine Feinplanung kommen hinzu. Die Masterarbeit soll klären, wie zuverlässig diese verteilte Datenkette im Sinne von ISO/IEC 25010 [10] ist, wo sie bei Fehlern Daten verliert oder verfälscht und welche Architektur für den weiteren Ausbau am besten geeignet ist.

2 Problemstellung

2.1 Das System als verteilte Datenkette

Abbildung 1 zeigt die bestehende Architektur. Ein Reader liest die Maschinen aus und schreibt die Daten in eine Datenbank. Ein Übertragungsmodul [Name] überträgt die Daten von dort in Cancom [Art des Systems und Schnittstelle klären]. Von Cancom gelangen die Daten zu einem API-Server, der sie mit MQTTnet [eingebetteter Broker oder Client eines externen Brokers?] an ein EIS und an einen Hallenspiegel verteilt. Als weiterer Verbraucher kommt eine Feinplanung hinzu.

Für die Zuverlässigkeit hat diese Kette zwei Folgen. Erstens fällt die Kette aus, sobald eine ihrer Stufen ausfällt. Sind die Ausfälle unabhängig, ist die Verfügbarkeit der ganzen Kette das Produkt der Verfügbarkeiten aller Stufen und damit geringer als die jeder einzelnen Stufe. Zweitens hat jede Übergabe ihre eigenen Garantien. Zwischen Datenbank, Übertragungsmodul, Cancom und API-Server muss jeweils geregelt sein, was bei einem Ausfall mit gerade übertragenen Daten geschieht. Wie sich die Kette bei Ausfällen einzelner Stufen verhält, ist bisher nicht systematisch untersucht. Mit jeder neuen Komponente wie der Feinplanung wird diese Frage wichtiger.

2.2 Was MQTT garantiert und was nicht

MQTT bietet drei Dienstgüten (Quality of Service, QoS) [5], [11]. QoS 0 stellt höchstens einmal zu, QoS 1 mindestens einmal und QoS 2 genau einmal. Diese Garantien gelten aber

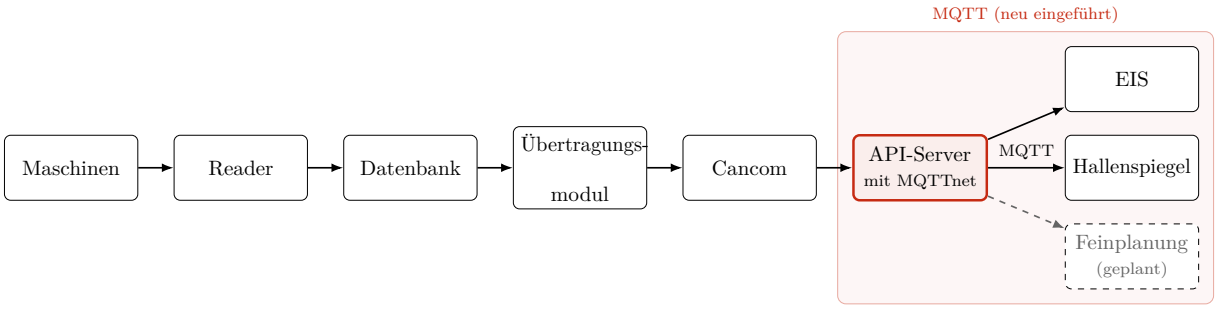


Abbildung 1: Bestehende Architektur bei der ProSeS GmbH (vereinfacht). Die Daten durchlaufen sechs Stufen, bevor sie beim Verbraucher ankommen. MQTT wird nur auf der letzten Strecke eingesetzt.

jeweils nur für eine Strecke, also vom Publisher zum Broker und getrennt davon vom Broker zum Subscriber. Ob eine Nachricht beim Verbraucher tatsächlich verarbeitet wurde, ist damit nicht gesichert. Die Reihenfolge wird nur innerhalb eines Topics und einer Dienstgüte garantiert, bei Shared Subscriptions in MQTT 5 auch das nicht.

Für den Wiederanlauf gibt es zwei Mechanismen. *Retained Messages* speichern pro Topic genau die letzte Nachricht, und *persistente Sessions* puffern Nachrichten für einen getrennten Client. Beide sind kein Verlauf. Wer nach einem Ausfall wissen will, was in der Zwischenzeit passiert ist, bekommt bestenfalls den letzten Wert, und auch das nur, solange Puffergrenzen und Ablaufzeiten des Brokers nicht überschritten sind. Wie ein Broker seine Daten persistiert, ist nicht standardisiert. Ein mit MQTTnet eingebetteter Broker hält Sessions und Retained Messages zunächst nur im Arbeitsspeicher, die Persistenz muss die Anwendung selbst umsetzen.

2.3 Was die Verbraucher brauchen

Hallenspiegel, EIS und Feinplanung leiten aus den Meldungen einen Zustand ab, zum Beispiel den Status jeder Maschine oder die gefertigte Stückzahl eines Auftrags. Damit alle Verbraucher denselben, korrekten Zustand sehen und ihn nach einem Fehler wiederherstellen können, muss er *deterministisch* aus den Meldungen folgen, also bei gleicher Eingabe immer gleich sein [12]. Dafür braucht es über alle Stufen hinweg eine eindeutige Reihenfolge der Meldungen [13], eine idempotente Verarbeitung von Duplikaten und an jeder Übergabe einen Punkt, ab dem nach einem Ausfall weitergemacht werden kann [14], [15].

2.4 Zuverlässigkeit als Bewertungsrahmen

Um Architekturen vergleichbar zu machen, wird Zuverlässigkeit (Reliability) nach ISO/IEC 25010 [10] als Bewertungsrahmen verwendet. Zuverlässigkeit ist dort der Grad, in dem ein System festgelegte Funktionen unter festgelegten Bedingungen über einen festgelegten Zeitraum erfüllt. Die Norm unterteilt sie in vier Teilmerkmale. Die folgende Tabelle überträgt sie auf die Datenkette und nennt mögliche Messgrößen. Fehlerbegriffe und Fehlerklassen werden ergänzend aus der Taxonomie von Avižienis et al. [16] übernommen.

Teilmerkmal	Bedeutung für die Datenkette	Mögliche Messgrößen
Fehlerfreiheit (Faultlessness)	Im Normalbetrieb kommt jede Maschinenmeldung genau einmal und in der richtigen Reihenfolge bei allen Verbrauchern an.	Lücken, Duplikate und Umordnungen ohne eingebaute Fehler
Verfügbarkeit (Availability)	Hallenspiegel, EIS und Feinplanung erhalten aktuelle Daten, wenn sie sie brauchen.	Anteil der Zeit mit aktuellen Daten, Datenalter beim Verbraucher
Fehlertoleranz (Fault tolerance)	Der Ausfall einer Stufe, etwa der Datenbank, von Cancom oder des API-Servers, führt nicht zu Verlust, Verdopplung oder Umordnung.	Verlust und Duplikate unter eingebauten Fehlern, aufgeschlüsselt nach Stufe
Wiederherstellbarkeit (Recoverability)	Nach einem Ausfall werden fehlende Daten nachgeholt, und alle Verbraucher erreichen wieder den korrekten Zustand.	Zeit bis zur Wiederherstellung (RTO), Umfang verlorener Meldungen (RPO), Abweichung vom Sollzustand

Bisher werden MQTT-Architekturen meist nur nach einzelnen Mechanismen wie QoS-Stufen oder Clustering bewertet. Ob eine mehrstufige Kette in allen vier Teilmerkmalen zuverlässig ist, wird selten zusammenhängend betrachtet. So kann QoS 1 auf der letzten Strecke die Fehlertoleranz gegen Verlust erhöhen und gleichzeitig durch Duplikate die Fehlerfreiheit beim Verbraucher gefährden, wenn dieser nicht idempotent arbeitet.

2.5 Die Lücke

Zwischen den Garantien einzelner Übergaben und den Anforderungen der Verbraucher gibt es eine Lücke, die in der Praxis meist anwendungsspezifisch und oft ungeprüft geschlossen wird. Es fehlt eine zusammenhängende Darstellung,

- welche Fehler in mehrstufigen, MQTT-basierten IIoT-Datenketten tatsächlich auftreten, an welcher Stufe sie entstehen und welches Teilmerkmal der Zuverlässigkeit sie betreffen,
- welche davon von MQTT, dem Broker oder den einzelnen Komponenten abgefangen werden und welche nicht,
- und welche Architektur unter welchen Fehlern von der Maschine bis zum Verbraucher fehlerfrei, verfügbar, fehlertolerant und wiederherstellbar bleibt.

Abgrenzung. Die Arbeit betrachtet die Zuverlässigkeit nach ISO/IEC 25010 von der Maschine bis zum Verbraucher. Leistungseffizienz wird nur als Aufwand der jeweiligen Architektur erfasst, etwa als zusätzliche Latenz oder Speicherbedarf, und nicht eigenständig optimiert. IT-Sicherheit (Authentifizierung, Verschlüsselung, Angriffe) ist nicht Gegenstand der Arbeit, ebenso wenig harte Echtzeitanforderungen auf Feldebene und die fachliche Logik von EIS, Hallenspiegel und Feinplanung. Andere Protokolle wie OPC UA werden nur als Quelle der Daten berücksichtigt, nicht als Alternative zu MQTT verglichen. Architekturvarianten werden als Prototypen in einer Testumgebung umgesetzt, nicht im laufenden Produktivbetrieb.

3 Forschungsfragen und Ziele

Ziel der Arbeit ist es, den Stand der Technik zur Zuverlässigkeit mehrstufiger, MQTT-basierter Datenketten aufzuarbeiten, die relevanten Fehler in einem Fehlermodell festzuhalten und die bestehende Architektur sowie Alternativen anhand der vier Teilmerkmale der Zuverlässigkeit zu messen. Daraus ergeben sich drei Forschungsfragen.

FF1 Welche Ansätze für eine zuverlässige Übertragung von der Quelle bis zum Verbraucher beschreibt die Literatur für mehrstufige, MQTT-basierte IIoT-Architekturen, und welche Teilmerkmale der Zuverlässigkeit sichern sie ab?

FF2 Welche Fehler treten an welcher Stufe einer solchen Datenkette auf, welches Teilmerkmal der Zuverlässigkeit betreffen sie, und wie wirken sie sich auf den Zustand bei den Verbrauchern aus?

FF3 Wie fehlerfrei, verfügbar, fehlertolerant und wiederherstellbar ist die bestehende Datenkette im Vergleich zu alternativen Architekturvarianten, und mit welchem Aufwand?

Die Arbeit liefert drei Ergebnisse. Erstens eine strukturierte Übersicht über den Stand der Technik und verwandte Arbeiten, geordnet nach den Teilmerkmalen der Zuverlässigkeit (FF1). Zweitens ein Fehlermodell, das für jede Stufe der Kette Fehlerarten, das betroffene Teilmerkmal und die Auswirkung bei den Verbrauchern beschreibt (FF2). Drittens einen Vergleich der bestehenden Architektur mit Alternativen, mit Messergebnissen je Teilmerkmal aus einer Testumgebung und einer Empfehlung für den weiteren Ausbau bei der ProSeS GmbH (FF3).

4 Nutzen für die ProSeS GmbH

Die Arbeit liefert der ProSeS GmbH eine belastbare Grundlage für Architekturentscheidungen beim weiteren Ausbau der MQTT-basierten Datenkette. Im Einzelnen:

- ein Fehlermodell der bestehenden Datenkette, mit dem sich Schwachstellen an jeder Stufe benennen lassen,
- eine Grundlage für die Einbindung neuer Komponenten wie der Feinplanung, ohne die Zuverlässigkeit der übrigen Verbraucher zu gefährden,
- einen messbasierten Vergleich von Architekturvarianten statt einer Entscheidung nach Erfahrung oder Herstellerangaben, mit Kennzahlen wie Wiederherstellungszeit und Ereignisverlust, die sich als Zuverlässigkeitsanforderungen an künftige Anlagen formulieren lassen,
- eine wiederverwendbare Testumgebung, mit der sich Broker, Konfigurationen und Updates vor dem Einsatz unter Fehlerbedingungen testen lassen,
- konkrete Handlungsempfehlungen, zum Beispiel zu QoS-Einstellungen, Session-Konfiguration, Puffergrößen, Wiederaufsetzpunkten an den Übergaben und Idempotenz in Verbrauchern.

Gerade weil Fehler im Datenfluss in der Fertigung oft erst spät bemerkt werden, etwa als falsche Kennzahl oder fehlende Rückmeldung, ist ein systematischer Blick auf diese Fehler auch für den Betrieb wertvoll.

5 Rahmenbedingungen

Unterstützung durch das Unternehmen. Für die Durchführung wäre Folgendes hilfreich:

- eine Ansprechperson als Betreuung im Unternehmen und regelmäßige kurze Abstimmungen,
- Einblick in die eingesetzte Architektur (Reader, Übertragungsmodul, Cancom, API-Server, Topic-Struktur, Datenmodell, Verbraucher),
- wenn möglich anonymisierte Beispieldaten oder Logs zu bisherigen Störungen, um das Fehlermodell an realen Fällen zu prüfen,
- Zugang zu einer Test- oder Entwicklungsumgebung bzw. die Freigabe, Broker-Software in einer eigenen Umgebung zu nutzen.

Kein Eingriff in die Produktion. Alle Fehlerexperimente finden ausschließlich in einer isolierten Testumgebung statt. Produktivsysteme werden nicht verändert.

Vertraulichkeit. Unternehmensinterne Informationen werden nur in Abstimmung mit der ProSeS GmbH verwendet. Bei Bedarf wird die Arbeit mit einem Sperrvermerk versehen. Die wissenschaftlichen Ergebnisse, also Literaturübersicht, Fehlermodell und Vergleich, werden so formuliert, dass sie ohne interne Details nachvollziehbar sind.

6 Zeitplan

Die Bearbeitungszeit beträgt sechs Monate.

Phase	Monat	Ziel	Artefakt
1 Einarbeitung	1	Bestehende Datenkette bei der ProSeS GmbH aufnehmen, Recherche planen	Architekturdokumentation, Recherche-Protokoll
2 State of the Art	1–2	Literatur und Standards auswerten	Literaturübersicht (FF1)
3 Fehlermodell	2–3	Fehlerorte, Fehlerarten und Auswirkungen festhalten	Fehlermodell (FF2)
4 Testumgebung	3–4	Bestehende Kette und Varianten nachbilden, Fehlerinjektion aufbauen	Lauffähige Testumgebung
5 Experimente	4–5	Fehlerfälle durchspielen und je Teilmerkmal der Zuverlässigkeit messen	Messergebnisse (FF3)
6 Auswertung	5	Vergleich und Empfehlung ableiten	Bewertung, Empfehlung
7 Ausarbeitung	5–6	Schriftliche Arbeit und Abschlusspräsentation	Masterarbeit, Code im Repository

Literaturverzeichnis

[1] B. Mishra und A. Kertesz, „The Use of MQTT in M2M and IoT Systems: A Survey,“ *IEEE Access*, Jg. 8, S. 201 071–201 086, 2020. DOI: 10.1109/ACCESS.2020.3035849.

[2] P. T. Eugster, P. A. Felber, R. Guerraoui und A.-M. Kermarrec, „The Many Faces of Publish/Subscribe,“ *ACM Computing Surveys*, Jg. 35, Nr. 2, S. 114–131, 2003. DOI: 10.1145/857076.857078.

[3] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[4] *IEC 62264-1:2013 – Enterprise-control system integration – Part 1: Models and terminology*, International Electrotechnical Commission, 2013.

[5] A. Banks, E. Briggs, K. Borgendale und R. Gupta, Hrsg., *MQTT Version 5.0*, OASIS Standard, OASIS, 2019. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[6] J. H. Saltzer, D. P. Reed und D. D. Clark, „End-to-End Arguments in System Design,“ *ACM Transactions on Computer Systems*, Jg. 2, Nr. 4, S. 277–288, 1984. DOI: 10.1145/357401.357402.

[7] *Sparkplug Specification, Version 3.0.0*, Eclipse Foundation, 2022. besucht am 27. Okt. 2026. Adresse: https://sparkplug.eclipse.org/specification/.

[8] M. Fowler. „Event Sourcing,“ besucht am 27. Okt. 2026. Adresse: https://martinfowler.com/eaaDev/EventSourcing.html.

[9] J. Kreps, N. Narkhede und J. Rao, „Kafka: A Distributed Messaging System for Log Processing,“ in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

[10] *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, 2023.

[11] A. Banks und R. Gupta, Hrsg., *MQTT Version 3.1.1*, OASIS Standard, OASIS, 2014. besucht am 27. Okt. 2026. Adresse: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html.

[12] F. B. Schneider, „Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," *ACM Computing Surveys*, Jg. 22, Nr. 4, S. 299–319, 1990. DOI: 10.1145/98163.98167.

[13] L. Lamport, „Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Jg. 21, Nr. 7, S. 558–565, 1978. DOI: 10.1145/359545.359563.

[14] E. N. Elnozahy, L. Alvisi, Y.-M. Wang und D. B. Johnson, „A Survey of Rollback-Recovery Protocols in Message-Passing Systems," *ACM Computing Surveys*, Jg. 34, Nr. 3, S. 375–408, 2002. DOI: 10.1145/568522.568525.

[15] K. M. Chandy und L. Lamport, „Distributed Snapshots: Determining Global States of Distributed Systems," *ACM Transactions on Computer Systems*, Jg. 3, Nr. 1, S. 63–75, 1985. DOI: 10.1145/214451.214456.

[16] A. Avižienis, J.-C. Laprie, B. Randell und C. Landwehr, „Basic Concepts and Taxonomy of Dependable and Secure Computing," *IEEE Transactions on Dependable and Secure Computing*, Jg. 1, Nr. 1, S. 11–33, 2004. DOI: 10.1109/TDSC.2004.2.